

Compiladores - Proyecto 2

Carranza Bueno Lucas, 202210073 (100%)
 Céspedes Zevallos Adrian, 202210088 (100%)
 Chavez Zapata Lenin, 202210090 (100%)

Abstract—El presente informe consta de la extensión de un compilador realizada en C++, donde se abarca desde el análisis sintáctico hasta la generación de código máquina para SVML

Index Terms—Compilador, sintaxis, gramática

I. INTRODUCCIÓN

EL proyecto tiene la implementación de la máquina virtual SVM, del parser y visitors del lenguaje IMP-FUN definido inicialmente por la siguiente gramática:

```
Program ::= VarDecList FunDecList
VarDecList ::= (VarDec)*
FunDecList ::= (FunDec)+
FunDec ::= "fun" Type id "(" [ParamDecList]" Body "endfun"
ParamDecList ::= Type id "(" Type id)*
VarDec ::= "var" Type VarList ";"
Type ::= id
VarList ::= id "(" id)*
StatementList ::= Stm ";" Stm)* ...
Stm ::= id "=" CExp |
        "print" "(" CExp ")" |
        "if" CExp "then" Body ["else" Body] "endif" |
        "while" CExp "do" Body "endwhile" |
        return "(" [CExp] ")"
CExp ::= ...
Factor ::= ... | id "(" [ArgList] ")"
ArgList ::= CExp "(" CExp)*
```

Fig. 1. IMP-FUN

En el presente reporte se presentarán los cambios más importantes de forma resumida. Con el objetivo de mantener el documento conciso y legible se han resumido los snippets de código mostrados, y algunos cambios fueron explicados de forma verbal.

Nota: en el código de la entrega se proveen ejemplos de código imp-fun que muestran la funcionalidad del compilador.

II. CÁLCULO DE MÁXIMA ALTURA DE PILA

En primer lugar, se requiere calcular la máxima altura de pila para una función. Se emplearon las funciones `sp_incr` y `sp_decr` provistas en el typechecker para esto. Cada expresión y statement tiene un impacto diferente:

- `AssignStm` y `PrintStm` consumen un valor, por lo que realizan `sp_decr(1)`.
- Tanto `IfStm` como `WhileStm` terminan consumiendo lo generado por el cond check, por lo que realizan `sp_decr(1)`.
- `ReturnStm` consume la expresión dentro de sí si es que hay, por eso puede realizar `sp_decr(1)`.
- `BinaryExp` consume dos operandos para generar uno, por lo que incluye `sp_decr(1)`.

- `NumberExp`, `TrueFalseExp` y `IdExp` añaden un elemento a la pila, por lo que hacen `sp_incr(1)`.
- `CondExp` consume el valor del condicional y realiza `sp_decr(1)`.
- `ParenthExp` no hace nada.
- `FCallExp` añade un elemento si la función retorna, por lo que puede hacer `sp_incr(1)`. Adicionalmente, `FCallExp` consume tantos valores como parámetros de función, por lo que realiza siempre `sp_decr(m)`, siendo m el número de parámetros.

Esto finalmente tiene efecto en `visit(FunDecList* s)`, donde se inicializa `max_sp = 0`, se calcula al realizar el visit del `FunDec`, y se almacena en el `FEntry` de la función.

```
void ImpTypeChecker::visit(FunDecList* s)
{
    for (it = s->fdlist.begin(); it !=
        s->fdlist.end(); ++it) {
        sp = max_sp = 0;
        dir = max_dir = 0;
        (*it)->accept(this);
        FEntry fentry;
        fentry.fname = (*it)->fname;
        fentry.ftype = env.lookup(fname);
        fentry.max_stack = max_sp;
        fentry.mem_locals = max_dir;
        //...
    }
}
```

Finalmente, se utilizará `fentry.max_stack` junto con el numero de variables locales en el codegen para calcular el valor de enter para cada función.

III. CONTEO DE VARIABLES LOCALES

Luego, se requiere almacenar en el environment las direcciones de las variables globales y locales, para permitir calcular el offset durante el codegen.

Para resolver el problema del número de variables locales por función, se utilizaron las variables `dir` y `max_dir` del typechecker. Estos números permiten contar el número de variables. Se incrementa `dir` con cada declaración de variable, y se reduce cuando alguna variable sale de scope.

La principal modificación se da al visitar una declaración de variable. Se incrementa en 1 el valor de `dir`, y si sobrepasa a `max_dir`, se sobrescribe.

```
void ImpTypeChecker::visit(VarDec* vd) {
    //...
    for (it = vd->vars.begin(); it !=
        vd->vars.end(); ++it) {
```

```

        env.add_var(*it, type);
        // actualizar dir y max_dir
        dir++;
        if (dir > max_dir)
            max_dir = dir;
    }
}

```

Por último, tenemos que considerar cuando las variables salen de scope, pues no queremos almacenar espacio innecesario para variables muertas. Cuando se cree un nuevo nivel al environment, guardamos el valor actual de dir, y lo restauramos al quitar el nivel. Todo cambio que se haya hecho a max_dir dentro del scope igual perdurará.

```

void ImpTypeChecker::visit(Body* b) {
    // guardar direccion actual (dir)
    int temp_dir = dir;

    env.add_level();
    b->var_decs->accept(this);
    b->slist->accept(this);
    env.remove_level();

    // restaurar direccion de entrada
    dir = temp_dir;
}

```

De la misma manera que hicimos con max_sp en el ítem anterior, inicializamos dir y max_dir en 0 antes de visitar una FunDec. Tras visitarla, guardamos el valor de max_dir en su FEntry respectivo para poder manejarlo en el codegen. (El código de visit FunDecList se puede ver en el ítem anterior).

```

void ImpCodeGen::visit(FunDec* fd) {
    //...
    codegen(get_flabel(fd->fname),
        ↪ "skip");
    codegen(nolabel, "enter",
        ↪ fentry.max_stack +
        ↪ fentry.mem_locals);
    codegen(nolabel, "alloc",
        ↪ fentry.mem_locals);
    //...
}

```

IV. DIRECCIONES DE VARIABLES Y RETORNO

Lo último que necesitamos para poder utilizar las variables locales en el codegen es un mapeo de las direcciones de estas.

Lo primero que hacemos es asignarle la dirección a los parámetros de función. Para ello utilizamos la clase VEntry en el codegen, asignando a parámetro el scope local y su offset correspondiente: $i - (m + 3)$. Hacemos lo mismo para el return, colocándolo en $-(m + 3)$

```

void ImpCodeGen::visit(FunDec* fd) {
    FEntry fentry =
        ↪ analysis->ftable.lookup(fd->fname);
    current_dir = 0;
    int m = fd->types.size();
    VarEntry ventry;

```

```

        // agregar direcciones de argumentos
        int i = 0
        for (auto it = fd->vars.begin(); it !=
            ↪ fd->vars.end(); ++it) {
            current_dir++;
            ventry.dir = i - (m+3);
            ventry.is_global = false;
            direcciones.add_var(*it, ventry);
        }
        // agregar direccion de return
        ventry.dir = -(m+3);
        ventry.is_global = process_global;
        direcciones.add_var("return", ventry);

        //...
    }

```

Por último, para trabajar con variables locales utilizamos el valor de current_dir, el cual tiene el valor de 0 antes de hacer el codegen del body de la función. Cada vez que se procesa una declaración de variable, se incrementa en 1 y se guarda esa dirección en el VEntry de la variable. Realizamos la misma consideración que el typechecker para el conteo de variables, y restauramos el valor previo de current_dir tras reducir un nivel en el environment.

```

void ImpCodeGen::visit(VarDec* vd) {
    list<string>::iterator it;
    for (it = vd->vars.begin(); it !=
        ↪ vd->vars.end(); ++it) {
        current_dir++;
        VarEntry ventry;
        ventry.dir = current_dir;
        ventry.is_global = process_global;
        direcciones.add_var(*it, ventry);
    }
}

void ImpCodeGen::visit(Body* b) {
    int temp_dir = current_dir;
    direcciones.add_level();

    //...

    direcciones.remove_level();
    current_dir = temp_dir;
}

```

V. SENTENCIA FCALLSTM

Esta sentencia se implementó de forma equivalente al FCallExp visto en clase, con la diferencia que se debe manejar el caso donde se descarta el valor de retorno.

- 1) **Parser:** se modificó el parser, expandiendo la función `parseStatement()`. Cuando se encuentra un id seguido por el token "(", se realiza el parsing de los parámetros y se retorna un statement del tipo `FCallStatement`.
- 2) **TypeChecker:** se valida de la misma manera que el typecheck de `FCallExp` realizado en clase. La principal diferencia ahora es que se realiza un `sp_decr(1)` adicional después de la llamada si es que la función retorna, puesto que se hace pop de este valor.

```
tcheck(env, FCallStm(fname, e1 :: ... :: env))
  if i
    env(fname) = (T1, ..., Tn) -> Tret
    tcheck(env, ei) = Ti
```

- 3) **Codegen:** se genera código de la misma manera que se vio en la teoría del curso para un FCall. La única diferencia es que se realiza un pop tras la llamada si es que la función retorna un valor, para así restaurar la pila a como estaba.

```
void ImpCodeGen::visit(FCallStm* s) {
    FEntry fentry =
        ↳ analysis->ftable.lookup(s->fname);
    ImpType ftype = fentry.ftype;

    if (ftype.ttype != ImpType::VOID)
        codegen(nolabel, "alloc", 1);

    list<Exp*>::iterator it;
    for (it = s->args.begin(); it !=
        ↳ s->args.end(); ++it) {
        (*it)->accept(this);
    }
    codegen(nolabel, "mark");
    codegen(nolabel, "pusha",
        ↳ get_flabel(s->fname));
    codegen(nolabel, "call");

    // descartar el valor de retorno
    if (ftype.ttype != ImpType::VOID)
        codegen(nolabel, "pop");
}
```

VI. SENTENCIA FORDOSTM

Esta sentencia se implementó de la siguiente manera:

- Se almacenan 2 variables locales por cada sentencia `ForDoStm`: el iterador y el tope. Sólomente el iterador es accesible al programador.
- Se almacena el tope en una variable por 2 motivos: número fijo de iteraciones y eficiencia.
- En el caso donde haya un `IdExp` en el `CExp` del tope, donde el valor de este id cambie a lo largo de la vida del bucle; el número de iteraciones sería inconsistente si se tuviera que evaluar el tope cada iteración.
- Al inicio de cada iteración, se evalúa si el iterador es mayor que el tope. En caso de que lo sea, se va al label del final, y si no, se continúa.
- Se suma 1 al iterador tras cada iteración.

- 1) **Parser:** se modificó el parser añadiendo los keywords "for", "in" y "endfor". Se expandió la función `parseStatement()`, añadiendo el caso para el token "for". Se espera recibir el orden de tokens válidos, para abrir el body tras el token "do" y cerrarlo con "endfor". Se retorna un statement de tipo `ForStatement(id, start, end, body)`.
- 2) **TypeChecker:** se valida que ambas expresiones dentro de los paréntesis sean del tipo `TINT`. Al igual que con el body, se crea un nuevo nivel del environment debido a las nuevas variables locales: iterador y tope. Por este motivo se almacena temporalmente dir, se le suma 2 por las 2 variables nuevas, y se restaura tras cerrar el for.

Finalmente, al evaluar la condición de parada o al sumar 1 al iterador, el stack crece por dos, e inmediatamente disminuye por dos. Tenemos que tener en cuenta este cambio por si se sobrepasa el `max_stack` previo. Por este motivo, realizamos la visita a las 2 expresiones, donde cada visita realiza `sp_incr(1)` (se incrementa por 2). Tras realizar estas 2 visitas, realizamos un `sp_decr(2)`, retornando el sp a su altura previa, pero habiendo modificando el `max_stack` si hace falta.

```
tcheck(env, ForStm(id, e1, e2, body))
  if i
    tc(env, e1) = tc(env, e2) = int
    && tc(env1, body)
  where
    env1 = env[id->int]
```

- 3) **Codegen:** en primer lugar, se debe evaluar las expresiones del inicio y el tope, y guardarse en su respectiva dirección. se genera un label "l1" al inicio del bucle, tras el cual se genera el código del checking de condición de parada, el cual puede hacer un salto al label "l2", que está debajo de todo. Tras el codegen del body, se suma 1 al iterador y se realiza un salto incondicional a "l1".

```
void ImpCodeGen::visit(ForStm* s) {
    string l1 = next_label();
    string l2 = next_label();

    int temp_dir = current_dir;
    direcciones.add_level();
```

```

// memoria para el iterador y el
→ tope
current_dir += 2;

VarEntry iter_entry;
iter_entry.dir = current_dir - 1;
iter_entry.is_global = false;
direcciones.add_var(s->id,
→ iter_entry);

string end_id = "_" + s->id +
→ "_end";
VarEntry end_entry;
end_entry.dir = current_dir;
end_entry.is_global = false;
direcciones.add_var(end_id,
→ end_entry);

// inicializar iterador y tope
s->start->accept(this);
codegen(nolabel, "storer",
→ direcciones.lookup(s->id).dir);
s->end->accept(this);
codegen(nolabel, "storer",
→ direcciones.lookup(end_id).dir);

// bucle
codegen(l1, "skip");

// cond check
codegen(nolabel, "loadr",
→ direcciones.lookup(s->id).dir);
codegen(nolabel, "loadr",
→ direcciones.lookup(end_id).dir);
codegen(nolabel, "le");
codegen(nolabel, "jmpz", l2);

s->body->accept(this);

// incrementar iterador por 1
codegen(nolabel, "loadr",
→ direcciones.lookup(s->id).dir);
codegen(nolabel, "push", 1);
codegen(nolabel, "add");
codegen(nolabel, "storer",
→ direcciones.lookup(s->id).dir);

codegen(nolabel, "goto", l1);

// fin del bucle
codegen(l2, "skip");

direcciones.remove_level();
current_dir = temp_dir;
return;
}

```

Nota: Se puede ver que se almacena el tope del iterador con el id "_[id]_end". De este motivo esta variable no es accesible por el programador, y no interferirá con las demás variables locales.